

Icon Editor

Executive Summary

Introduce a new G-based Icon Editor, that gives the have more sophisticated tools to create icons. The new Icon Editor should help save time and encourage users to create better icons for their VIs/CTLs, classes, palettes and libraries.

Problem (Why?)

The icon editor has not been reorganized in several LabVIEW releases and is completely out of date. To create a simple icon takes a long time. Complex icons require lots of effort, and the tools to be successful are not available. The standard LV response is that if you want complex icons, you can use a third-party icon editor and copy/paste icons from there into the LV icon editor. Icons have always been common in LabVIEW, but with the introduction of libraries in 8.0 and dynamic dispatching in 8.2, and a wealth of new VI types over the last few releases, we now have icons that are supposed to share themes, be modified based upon each other, etc, and an integrated icon editor that "understands" LabVIEW has become more desirable.

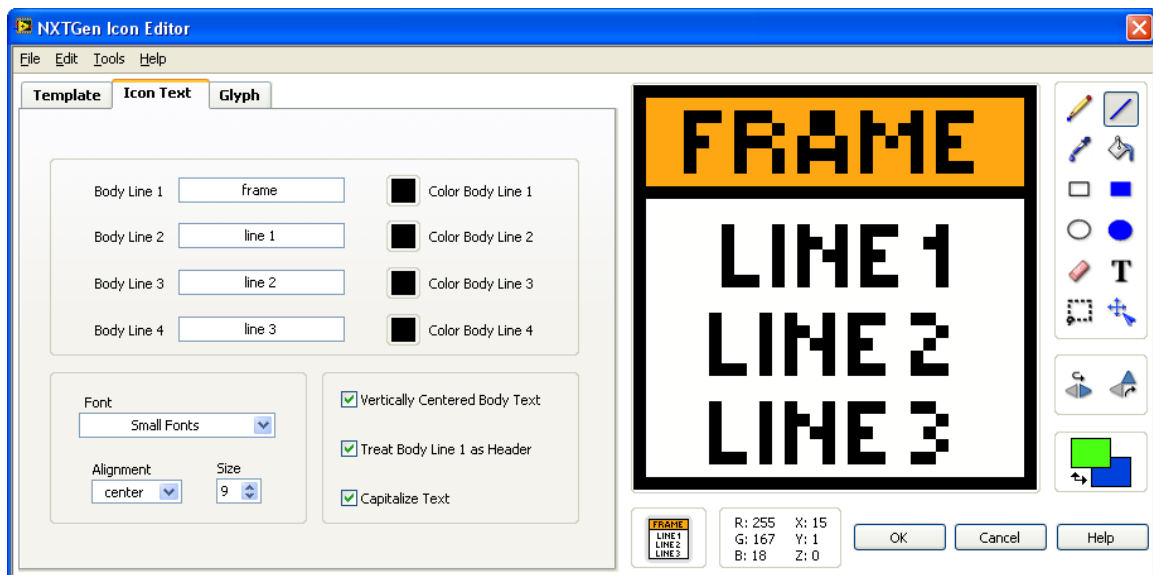
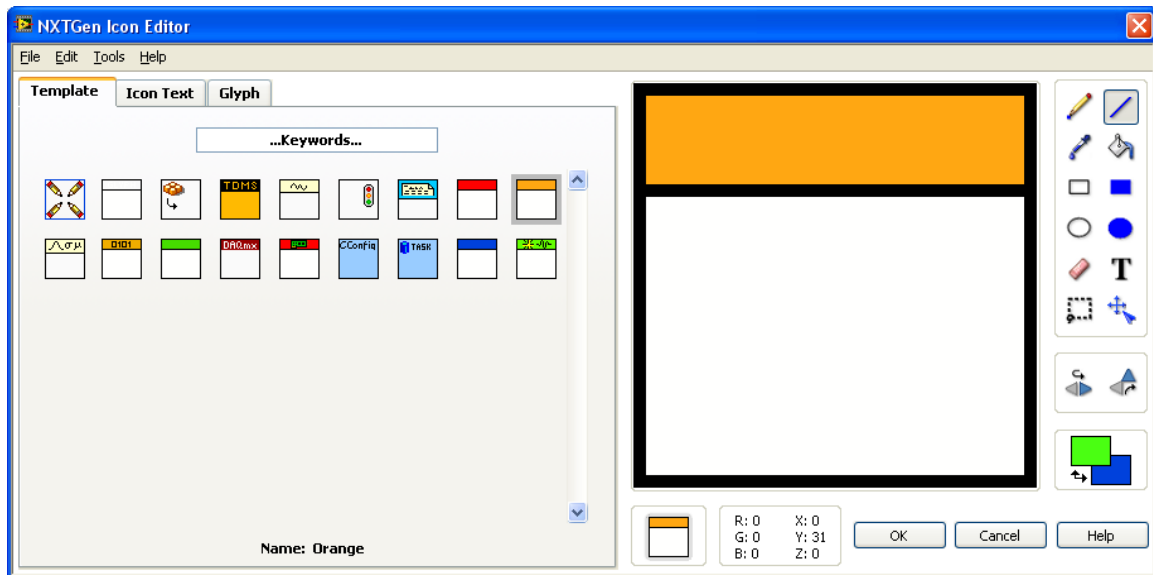
The recognition of a VI on the block diagram is based primarily on the icon, rather than on the VI label (which is by default not visible). Reading a block diagram should be as easy as reading a UML diagram or C code. By encouraging users to use the Icon Editor to create meaningful icons, that help to read and understand the code easily, rather than looking behind the scenes (VI name or block diagram). If we want to have our customers creating more meaningful icons, we need to provide a tool with this specific ability, whereas to create a meaningful icon should not take longer than a few seconds.

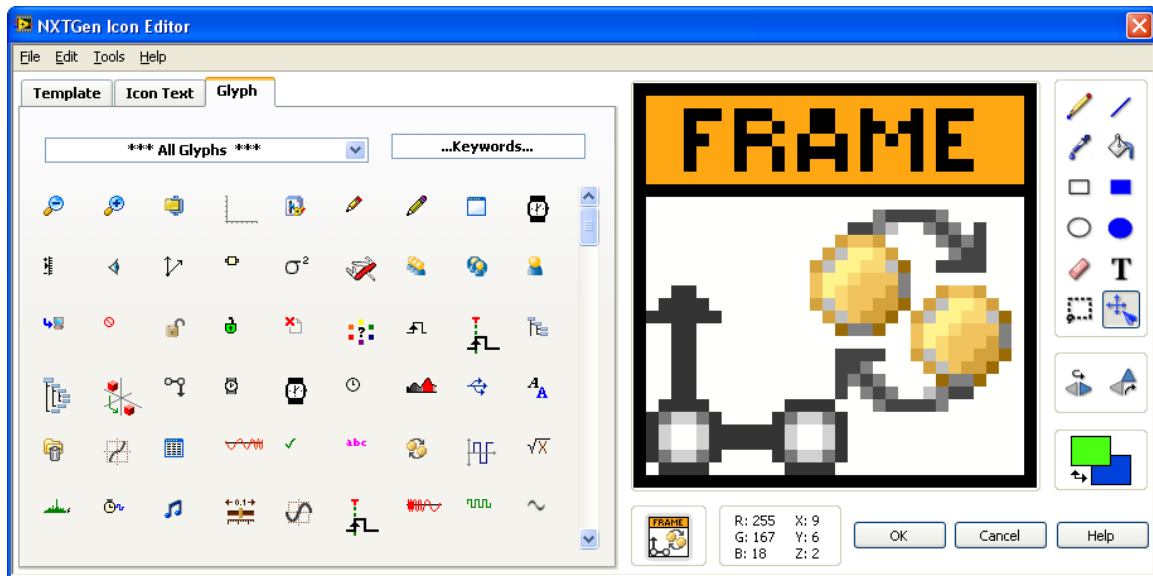
Solution (What?)

Description

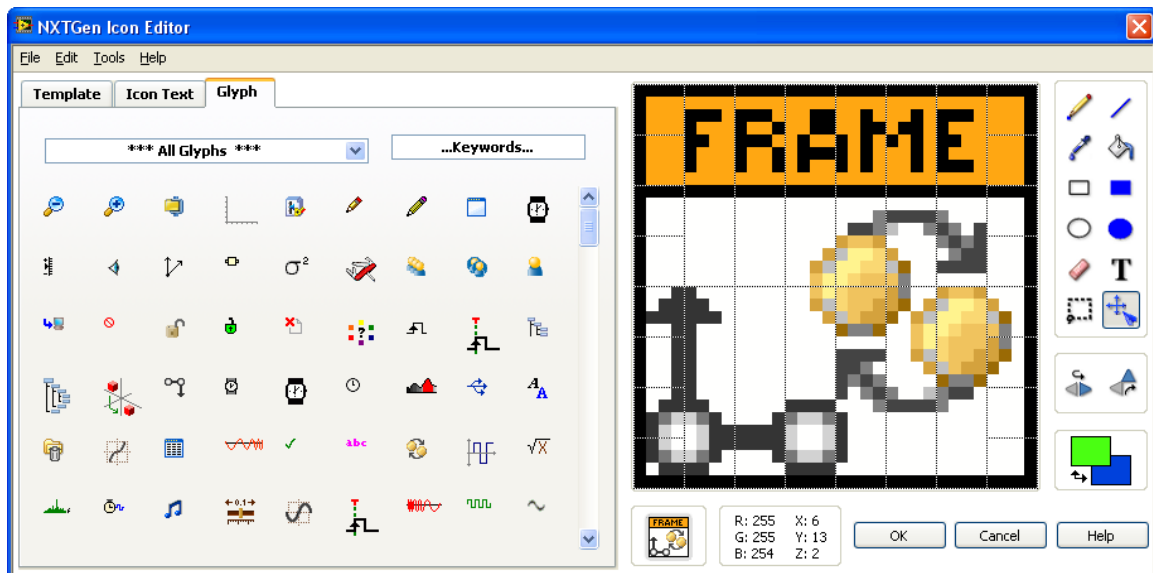
In recent weeks, I have had several meetings with <redacted> to get input based on the HMI, look and feeling as well as the functionality. A request from <redacted> was to program the new Icon Editor entirely in G with the option to add as easy as possible new features or to expand existing functionality. In addition to these meetings, I received customer input for features they would expect of an Icon Editor. These feature requests caused me to decide that the design of the current Icon Editor is obsolete and a new Icon Editor needs to be introduced with an entirely new look and feel.

Based on the input from usability experts, beginner and expert LabVIEW users, I created the following design:

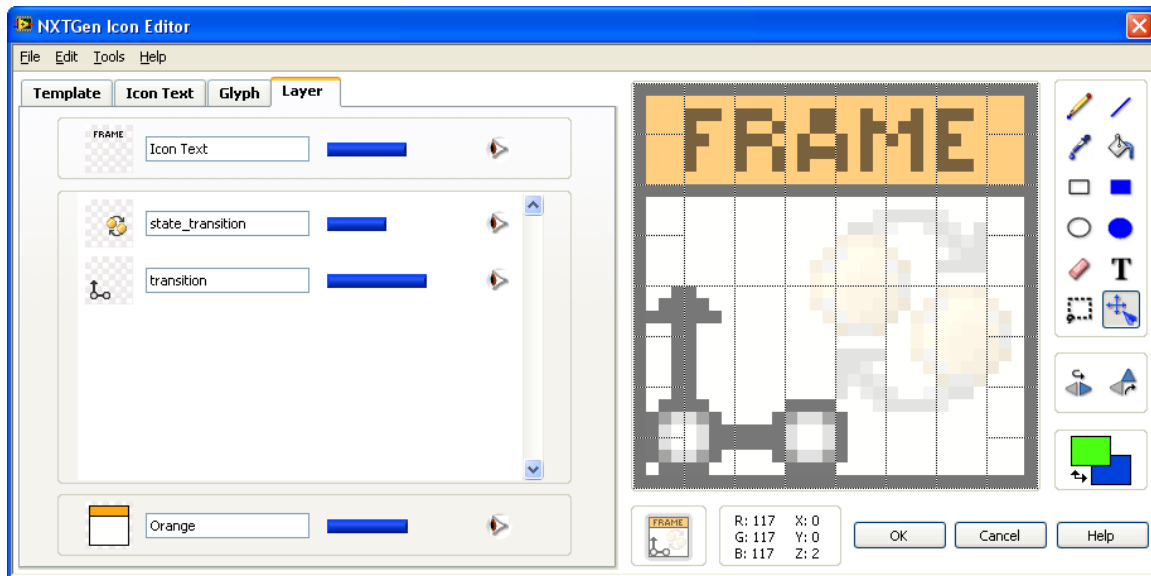




With visible terminals:



With visible terminals and layer tab:



This new icon editor maintains the existing functionality of the current Icon Editor. New features added include:

- Choice of an icon template, that can be used as base for the Library or VI icon
- An easy to use Icon Text option, to write text on the icon on predefined positions (with live update)
- Drag and Drop functionality of glyphs
- Multi Layer support (change layer order, change opacity and visibility of a layer)
- Rotate & Flip glyphs, layers or the entire icon
- 'Half Auto Update' of the glyphs from ni.com/iconlibrary
- Ability to create own templates and glyphs (support for PNG, BMP and JPEG format for templates and glyphs)
- Store the most recent settings in the LabVIEW.ini file
- # of Undo/Redo steps based on the Undo Steps in LabVIEW (source: LabVIEW.ini)
- Store all layers and a subset of the most recent settings per VI / Library
- 100% G-based

User Interface Impact

Tab Control

Tab:Template

The majority of users might want to start with a predefined icon template. The best internal examples to start the icon with a template are: DAQmx, CAN, FPGA, RT, Math library, TEDs, etc. The template library can be easily expanded by copying an existing graphic (max 32x32px) into a specific folder or by creating a new template on the fly out of the icon editor. Templates are used as a starting point for icon

design, eliminating the need to copy/paste templates from some source (another VI or another icon editor). With the keyword it is possible to search for specific templates by name (no additional search features are available at this time, such as searching for name synonyms, colors, etc).

The chosen template becomes the icon's background. It is a separate layer that is always the bottom layer. This background layer does show up in the list of layers (see [Tab:Layer](#) below). If no template is chosen, the layer is a plain, white background.

All templates have to be stored in a certain location on the disk. The root folder is read at every launch of the Icon Editor. All graphic files of a certain format in this folder (or its subfolders) are listed in the 'Template' list. By sharing VIs (user1 sends his VI to user2) it might happen, that user2 does not have the template on his disk. A VI stores all the layers used to build it, including the starting template. It is possible to extract the template layer and to save it on user's2 disk, so it is available as a template for future icon editing. The Icon Editor has options for controlling this feature:

- Save 3rd party templates: *boolean*
- (if the above is false, this is disabled) Save 3rd party templates into a specific folder: *path*

If the user does not want to save 3rd party templates on his disk, the template is only temporarily in the templates list.

The path to the specific 3rd party templates folder is added to the LabVIEW.ini file as the following:

```
[IconEditor]  
3rdPartyTemplates =
```

The keyword can be used to filter templates. All templates that have the same pattern (non case sensitive) in their filename (without the file extension) are shown in the templates list.

Tab:Icon Text

Users use this tab to easily write text on their icon.

On the icon itself a maximum number of 4 lines make sense. LabVIEW uses a specific, fixed font to type the text, which is not choosable by the user. Not all lines need to be used.

The text is by default auto-centered (vertically and horizontally) and will take into account icons with a header. The options "Vertically Centered Icon Text" and "Treat Body Line 1 as Header" can be used to adjust the position of the text on the icon. With the ability of the live update it is as easy as possible to correct or adjust

wording. Each body line may have its own color, but all lines share the same font, size and alignment. Like the template discussed above, the text of the icon is an independent layer of the icon that does appear in the Layers tab (see [Tab:Layer](#) below)

Tab:Glyph

The glyph library contains all on the disk available glyphs that are stored in a specific folder in the format *.png, *.jpg or *.bmp.

When the Icon Editor is loaded, a dynamic VI in background gathers and prepares all glyphs and present them afterwards in the glyph list. Each glyph is tied to a certain category. The UI allows the user to display all the glyphs, glyphs of a certain category or only glyphs that match the typed keyword(s).

To add a glyph to the icon, either a double click on the glyph or drag the glyph onto the icon. The double click on the glyph drops it in the upper left corner. A drag and drop action displays the glyph during hovering the mouse over the big icon display, giving a look/feel very similar to dropping an item onto the panel/diagram from the LabVIEW palette.

- Mouse button down - hold the mouse button - move the mouse - release the mouse button - drop the block
- Mouse button down - release the mouse button - move the mouse - mouse button down - drop the block

While the drag is ongoing, it is possible to rotate or flip the glyph by using the shortcut keys "R" or "r" for Rotate and "F" or "f" for Flip. To cancel the current drag & drop action, use the Escape key on the keyboard.

Each glyph always gets its own layer so that the user can move the specific glyph around, rotate it, flip it or delete it after it got dropped without affecting other glyphs. Enhanced options on the [Layer tab](#) provide the ability to change the opacity, among other options.

The structure of the glyphs on the disk is nested. The root structure is the main glyph folder with a bunch of subfolder. The subfolders represent the categories, whereas each category may contain certain glyphs. Concerning the indexing of the glyph root folder at every launch of the Icon Editor, all changes are reflected in the category and glyph list immediately.

- Glyph folder
 - Acquiring and Generating Data
 - Analog Input.png
 - Analog Output.png
 - etc
 - Attributes

- Color.png
 - Font.png
 - Font Size.png
 - etc
- Signals
 - Analog.png
 - Digital.png
 - etc

The keyword can be used to filter templates. All templates that have the same pattern (non case sensitive) in their filename (without the file extension) are shown in the templates list.

The Icon Editor gets shipped with the whole set of glyphs that are in the ni.com/iconlibrary.

Tab:Layer

The layer tab provides an overview of all layers. The order of the user layers can be changed by moving layers up or down. Furthermore, it is possible to change the opacity and the visibility of individual layers. Because layers are an advanced icon editing feature, this tab is not shown by default. To get access to all layers, the user needs to enable the option using **Tools>>Show Layer Palette**.

Tools

The drawing tools are located on the right-hand side of the Icon Editor.

Tool drawings (anything you draw using the tools palette) are, by default, placed in separate layers from the glyphs. So if you drop a glyph and then start drawing, the Icon Editor will automatically add a new layer for the drawing. Multiple consecutive uses of draw tools will all go in the same layer unless you explicitly create a new drawing layer or drop a glyph (which always goes in its own layer).

Tool actions are always related to the current active color (top color box), except for the movement of a selected area. In this case the specified color in the bottom color box is used to fill the original position of the selected area.



Rectangle	Used to draw outlines of rectangles on the icon. A double-click draws the border of the icon.	
Filled Rectangle	Used to draw filled-in rectangles on the icon. A double-click draws a solid rectangle over the entire icon.	
Circle	Used to draw outlines of circles on the icon.	
Filled Circle	Used to draw filled-in circles on the icon.	
Line	Used to draw a single pixel, non-anti-aliased line on the icon.	
Pencil	Used to draw arbitrary scribbles on the icon (single pixel, non-anti-aliased)	
Color Select	Used to choose the current draw colors by selecting a pixel already on the icon and using that pixel's color as one of the draw	

colors. A left click is used to select the new fill color. A right click is used to set the new edge color.

Behavior with layers:

If there is no active layer then the color selected is the color of the pixel when all layers are combined. So, for example, if the top layer is blue but is set to 50% transparent and the bottom layer is red, then the color selected will be the resulting shade of purple. If there is an active layer then the color selected will be the color of the pixel on the active layer or the background color if the selected pixel is transparent on the active layer.

This tool paints a region of the icon with the current draw color. The region is all of the pixels contiguous to the clicked pixel that are of the exact same color as the clicked pixel.

Fill

Behavior with layers:

If no layer is active, the color of a pixel is judged by the combined color of all of the visible layers, not just the color of the pixel as it is on the the current drawing layer. If an active layer is selected, the color of a pixel is the color of that pixel on the active layer only.



The eraser tool behaves like the pencil tool except that it sets the color of the pixels to transparent.

Eraser

Behavior with layers:

If there is no active layer when this tool is used, then the pixel is erased on all editable layers of the icon, so the template and text layers will show through. If there is an active layer selected, the pixel is erased only on the active layer, allowing any lower layer to show through.



Selection	Drag to select a rectangular region of the icon. Once a region is selected, the Delete key may be used to delete the selected pixels (equivalent to using the Eraser tool on the entire region) or the selected pixels may be dragged to a new location within the icon. Whenever a selection is made, the CTRL C only considers the selected area. If the selected area is made on a selected layer only, only the pixels of the selected layer are copied or moved. If there is no selected layer, all layers (exclusive icon text layer) are copied. When you paste, if multiple layers were copied, they paste as a single image. The template and text layers are included in any selected region for copying, but not for moving. Behavior with layers: If there is no active layer, the pixels on all layers are selected. If there is an active layer, then only pixels on the active layer are selected.	
Text	Used to put text on the icon. Double clicking opens the Font dialog to allow the user to select font size and style.	
Layer Selection	The Layer Select tool allows the user to choose the active layer. The active layer is any of the editable layers, meaning any of layers other than the templay layer and the text layer. Those two layers can never be selected. To use this tool, the user clicks on the tool and then clicks on a pixel in the icon. The first layer (other than the template or text layers) that has a non-transparent pixel in that position becomes the active layer, and all other layers become temporarily transparent. To deselect the layer, the user can press the ESC key or can click on the	

Layer Select tool again and just pick a pixel that has no color on any editable layer. When there is no active layer, all the temporarily transparent layers return to visibility.

If a layer is active, all other tools effect the active layer only.

When pixels are selected, the behavior of this tool is to perform a horizontal flip on the selected pixels.

Horizontal Flip

When no pixels are selected, the behavior varies depending upon whether there is an active layer or not.

Behavior with layers:

If there is no active layer, "the editable layers" refers to all layers except the template and text layers.

If there is an active layer, "the editable layers" is the active layer only.

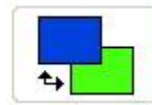


Clockwise Rotate

Same as flip except it performs a clockwise rotation on the pixels.



The color boxes are used to set the current fill and edge colors. The fill color (shown here as blue) is used by most of the drawing tools. The edge color (shown here as green) is used only by the filled circle and filled rectangle tools to draw the outline of their shapes.



Menu Bar

The menu bar comprises most of the menu tags of the old Icon Editor with some new items

- _File
 - _Save Icon as...
 - Template Icon
 - -----
 - PNG Format
 - Close (CTRL W)
- _Edit
 - Undo (CTRL Z)
 - Redo (CTRL Shift Z)

- -----
 - Copy to Clipboard (CTRL C)
 - Paste from Clipboard (CTRL V)
 - -----
 - Select All (CTRL A)
 - -----
 - Select Font...
 - -----
 - Show Terminals (CTRL T)
- _Tools
 - Synchronize With ni.com Icon Library
 - -----
 - List Glyphs and Icon Templates
 - -----
 - Icon Editor Settings
- _Layer
 - Show Layer Tab (CTRL L)
 - -----
 - Combine Selected Layers
 - Combine All Layers
- _Help
 - Icon Editor Help
 - -----
 - Show Context Help
 - Lock Context Help
 - -----
 - Search the LabVIEW Help...

Other UI Elements

- Picture Control 320x320px
 - Big Preview
 - The big preview is a picture control with a dimension of 320 x 320 pixels. This control is the bottom most of in total three super-imposed picture controls. The zoom feature is realized by enabling using the ZoomFactor attribute of the picture control itself.
- Terminals
 - An other picture control is used to draw the terminals of the VI. This picture control lays on top of the Big Preview picture control. The ZoomFactor of this picture control is 0.
- Selection
 - To fake the selected area with running ants, a separate picture control is used. This picture control lays on top of the Terminals picture control (top most) and its ZoomFactor is 0.

- Preview Picture Control 32x32px
 - The tiny preview picture control displays the real size and appearance of the icon
- Current Pixel Information
 - The information field contains data related to the Big Preview control.
 - The range of the X and Y mouse position is from 0..31
 - The Z (stands for the layers) is the number of all layers
 - R,G and B represents the color value of the pixel at the current mouse position.
- OK button
- Cancel button
- Help button

Icon Editor configuration data

Certain most recent values are stored in the LabVIEW.ini file to be used the next time the Icon Editor is launched. The following values/settings are saved in the LabVIEW.ini file, section [IconEditor]

- Tab:Icon Text -> Text color line 1 (*int*)
- Tab:Icon Text -> Text color line 2 (*int*)
- Tab:Icon Text -> Text color line 3 (*int*)
- Tab:Icon Text -> Text color line 4 (*int*)
- Tab:Icon Text -> Vertically centered Body Text, Treat Body Line 1 as Header and Capitalize Text (*int*)
- Tab:Icon Text -> Text font (*string*)
- Tab:Icon Text -> Text size (*int*)
- Tab:Icon Text -> Text alignment (*string*)
- Layer palette -> visible/invisible (*boolean*)
- Terminals -> visible/invisible (*boolean*)
- Most recent active Tab (*int*)
- Glyph category (*int*)
- Fill color (*int*)
- Edge color (*int*)
- Most recent active tool (*int*)

Feature Alternatives

- **This Icon Editor does not replace the LabVIEW Application Builder Icon Editor.** We talked about using this new icon editor for all of LV's icon editing needs, which would mean using this icon editor for creating the icons for built applications (EXEs). We decided that editor will continue to be the icon editor that ships with CVI. This new icon editor is intended for the icons for VIs and Libraries, and is very context aware when creating default icons. It does not support the different icon sizes nor all of the different file types that are needed for

built applications. Folding those two very different icon editor requirements together was something we decided not to tackle.

Internal Documentation (How?)

Detailed Design

This will be a G based dialog that will replace the existing icon editor using the plug-in mechanism added to LV in 8.0. (See LV online help regarding replacing the icon editor with a custom icon editor).

The only change to VIs themselves is that we will be saving the layer information of the icon in a VI Tag. This information will include

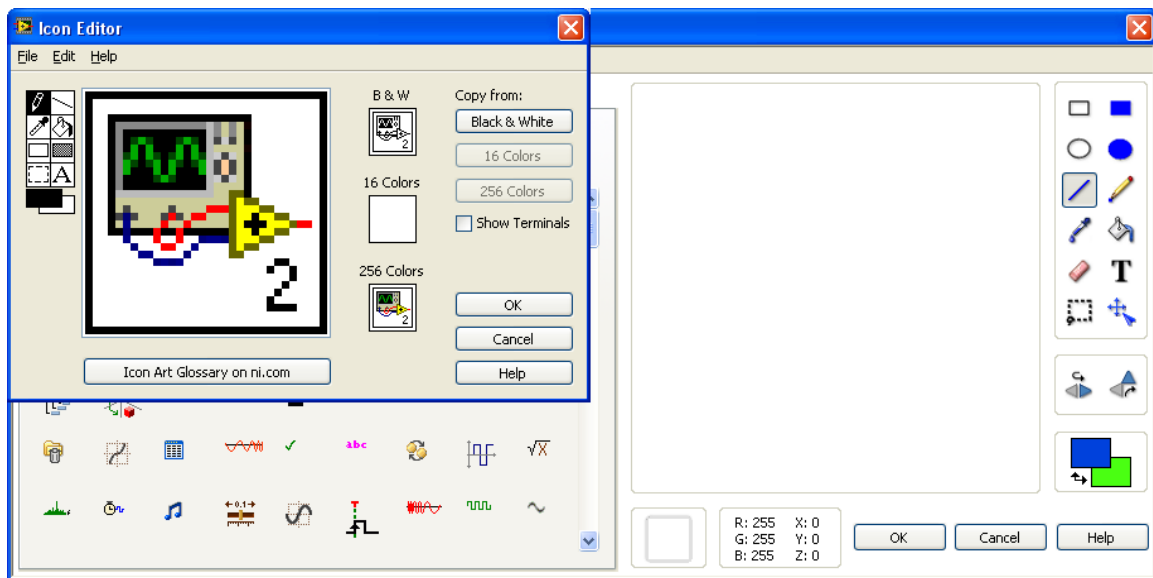
- which template was used for the background layer so that if the template changes the VI can update its icon the next time you edit the icon.
- the string used on the text layer, with the settings of the text (header, colors, font, etc)
- the separate layers of the icon, their opacity, etc,

All of this is stored off in a single VI Tag so that the next time the icon editor is used the icon can be restored from the last editing state. The Tag is not removed when saving the VI for a previous version.

We would like to have VIs update to match the template as part of loading, but that will have to wait for another version of LV or someone to have spare time late in the Orion release cycle.

The Icon Editor window is modal as well as the configuration window for the icon editor (that one is not covered in this spec right now). None of these dialogs will be resizable. The size of Icon Editor settings dialog is definitely smaller than 1024 x 768.

The Icon Editor itself right now has a size of about 914 x 454 (the old icon editor size is 463 x 312). Below is a direct comparison of the size of the old and the new Icon Editor.



The icon editor does no longer support the 4 and 1bit (BW) icon explicitly. In background the BW icon is still produced but is not displayed to the user any more.